

PRÀCTICA 1: INTRODUCCIÓ A L'ENTORN VIVADO

Jordi Garcia
Mateu Magem

1. CREACIÓ D'UN NOU PROJECTE

New Project

Project Name
Enter a name for your project and specify a directory where the project data files will be stored.

Project name:

Project location:

☒ Create project subdirectory

Project will be created at: C:/.../OneDrive/Escritori/Q5/Sistemes Digitals/Prctiques S.Digitals/Pràctica1

Primer de tot creem un projecte nou. Un cop creat, creem un fitxer amb format VHDL i seleccionem la FPGA amb la que treballa la nostra placa.

New Project

Default Part
Choose a default Xilinx part or board for your project.

Parts | Boards

[Reset All Filters](#)

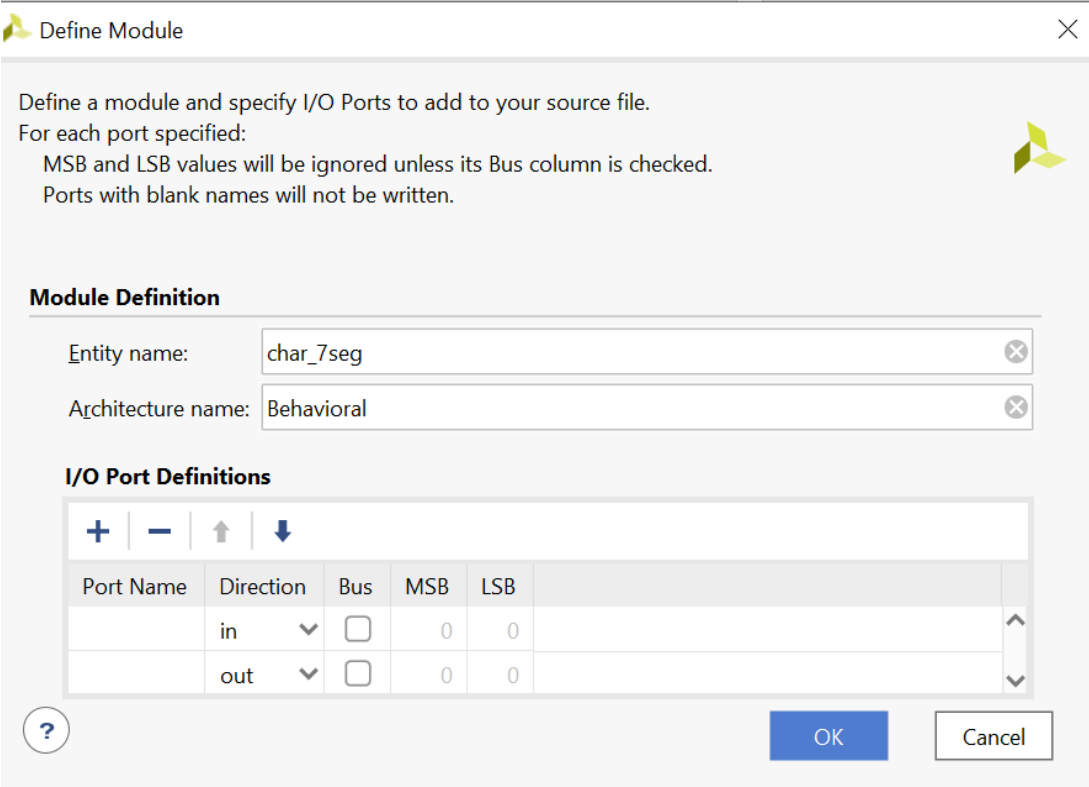
Category: Package: Temperature:

Family: Speed: Static power:

Search:

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs	Gb Transceivers	GTPE2 T
xc7a15tcpg236-1	236	106	10400	20800	25	0	45	2	2
xc7a35tcpg236-1	236	106	20800	41600	50	0	90	2	2
xc7a50tcpg236-1	236	106	32600	65200	75	0	120	2	2

Abans d'arribar a l'entorn de treball de Vivado definim els ports d'entrada i sortida de la nostra entitat juntament amb el seu número de bits.



Define Module

Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

Entity name: char_7seg

Architecture name: Behavioral

I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
	in	☐	0	0
	out	☐	0	0

OK Cancel

2. DESCRIPCIÓ DE L'ARQUITECTURA

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity char_7seg is
5      Port ( bin_in : in STD_LOGIC_VECTOR (3 downto 0);
6            cat : out STD_LOGIC_VECTOR (7 downto 0);
7            an : out STD_LOGIC_VECTOR (3 downto 0));
8  end char_7seg;
9
10 architecture Behavioral of char_7seg is
11
12 begin
13     with bin_in select
14         cat <= "00000011" when "0000", --0
15              "10011111" when "0001", --1
16              "00100101" when "0010", --2
17              "00001101" when "0011", --3
18              "10011001" when "0100", --4
19              "01001001" when "0101", --5
20              "01000001" when "0110", --6
21              "00011111" when "0111", --7
22              "00000001" when "1000", --8
23              "00011001" when "1001", --9
24              "00000101" when "1010", --a
25              "11000001" when "1011", --b
26              "11100101" when "1100", --c
27              "10000101" when "1101", --d
28              "01100001" when "1110", --E
29              "01110001" when "1111", --F
30              "11111111" when others;
31     an <= "1110";
32
33 end Behavioral;

```

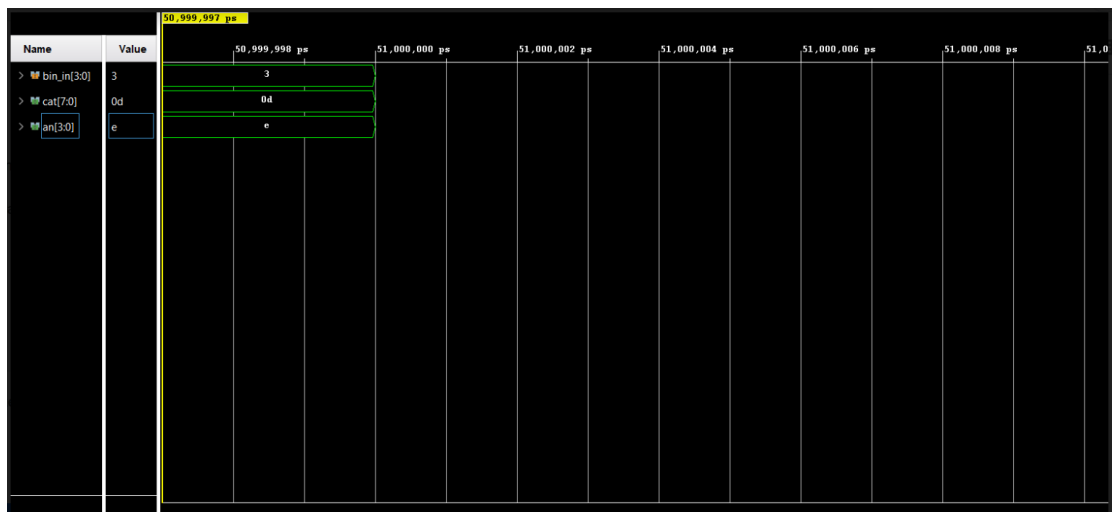
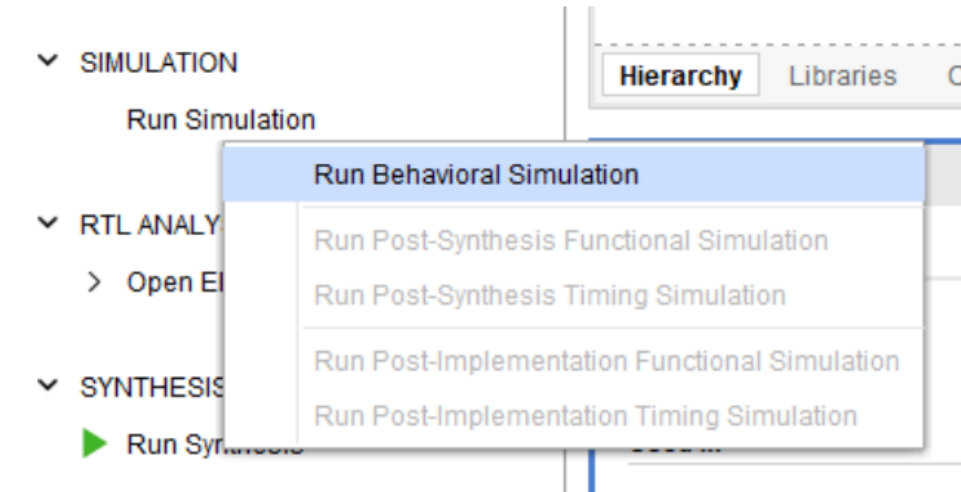
Un cop creat el projecte, el mòdul VHDL i l'entitat, toca programar l'arquitectura del nostre projecte. Com veiem a la imatge assignarem un valor en binari als càtodes de sortida en funció del valor de bin_in que introduïrem pels switch de la placa.

3. SIMULACIÓ

Podem realitzar dos tipus de simulació, la simulació directe i la simulació a partir d'un test bench. La simulació directe es fa directament des del nostre mòdul i hem de forçar manualment el valor de les entrades per veure com canvia la sortida.

La simulació test-bench canvia de forma predeterminada el valor de les entrades cada cert temps.

3.1 SIMULACIÓ DIRECTE



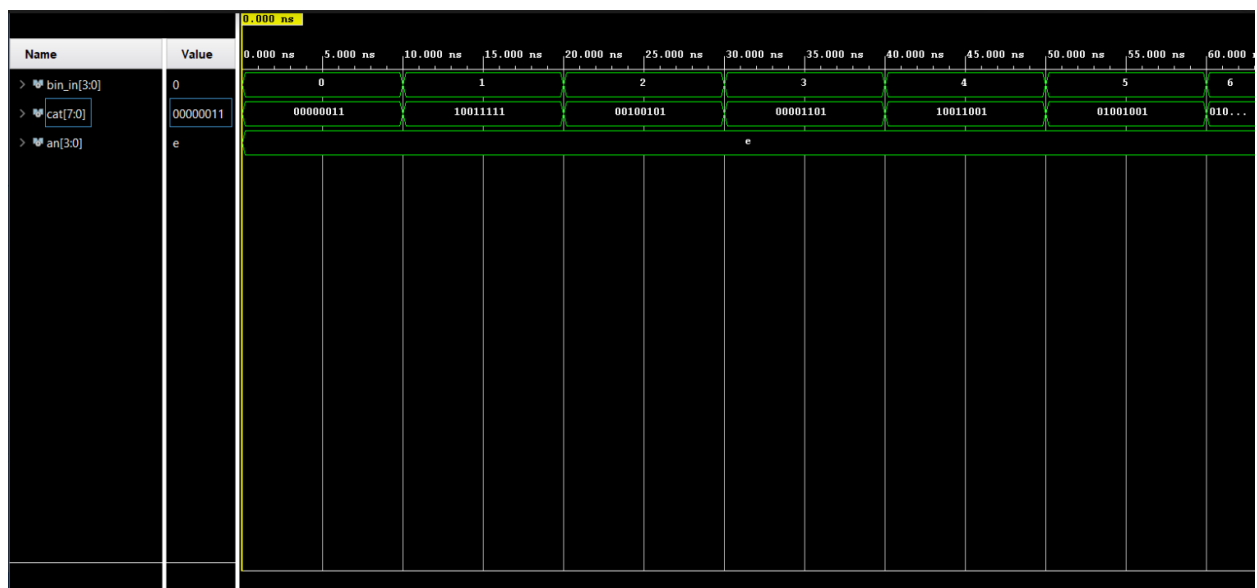
3.2 SIMULACIÓ TEST-BENCH

Per realitzar el test bench primer hem de crear un arxiu font de simulació. Un cop creat escrivim el codi que ens permetrà canviar el valor de l'entrada mitjançant un loop dins d'un procés.

```

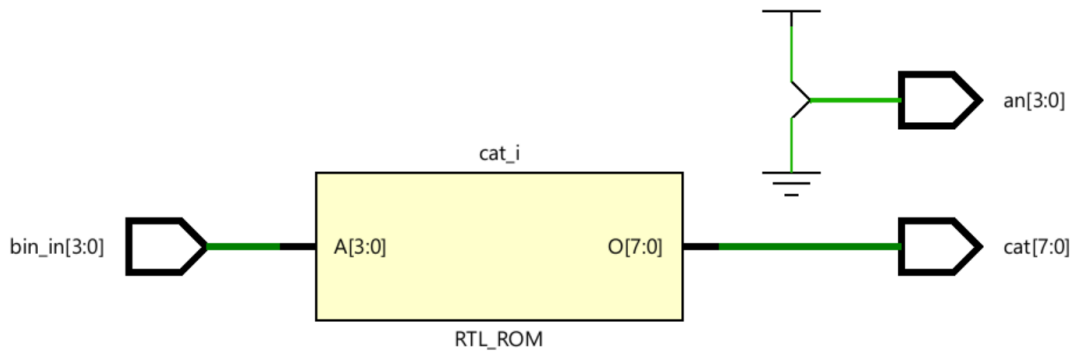
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity char_7seg_tb is
6  -- Port ( );
7  end char_7seg_tb;
8
9  architecture Behavioral of char_7seg_tb is
10
11  signal bin_in: std_logic_vector(3 downto 0);
12  signal cat: std_logic_vector(7 downto 0);
13  signal an: std_logic_vector(3 downto 0);
14
15  component char_7seg
16  Port ( bin_in : in STD_LOGIC_VECTOR (3 downto 0);
17        cat : out STD_LOGIC_VECTOR (7 downto 0);
18        an : out STD_LOGIC_VECTOR (3 downto 0));
19  end component;
20
21  begin
22
23  uut: char_7seg port map ( bin_in => bin_in, cat => cat, an => an);
24
25  test_entrades: process begin
26  for i in 0 to 15 loop
27      bin_in <= std_logic_vector(to_unsigned(i,4));
28      wait for 10 ns;
29  end loop;
30  end process;
31
32  end Behavioral;

```



4. RTL ANALYSIS

En aquest apartat es mostra el disseny a nivell RTL, és a dir, a nivell funcional. Podem veure el que Vivado ha interpretat del nostre codi en forma de esquema de blocs.



En aquest punt, podem assignar els ports del nostre mòdul a pins físics de la FPGA. Per fer-ho creem un arxiu de “constrains” i assignem els nostres ports a pins de la FPGA en funció del document Basys 3 FPGA Board Reference Manual

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
an (4)	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
an[3]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
an[2]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
an[1]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
an[0]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
bin_in (4)	IN					default (LVCMOS18)	1.800				NONE	NONE	
bin_in[3]	IN					default (LVCMOS18)	1.800				NONE	NONE	
bin_in[2]	IN					default (LVCMOS18)	1.800				NONE	NONE	
bin_in[1]	IN					default (LVCMOS18)	1.800				NONE	NONE	
bin_in[0]	IN					default (LVCMOS18)	1.800				NONE	NONE	
cat (8)	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
cat[7]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
cat[6]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
cat[5]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
cat[4]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
cat[3]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
cat[2]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
cat[1]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	
cat[0]	OUT					default (LVCMOS18)	1.800		12		NONE	FP_VTT_50	

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Termination	IN_TERM
an (4)	OUT				34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
an[3]	OUT		W4		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
an[2]	OUT		V4		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
an[1]	OUT		U4		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
an[0]	OUT		U2		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
bin_in (4)	IN				14	LVCMOS33*	3.300				NONE	NONE	
bin_in[3]	IN		W17		14	LVCMOS33*	3.300				NONE	NONE	
bin_in[2]	IN		W16		14	LVCMOS33*	3.300				NONE	NONE	
bin_in[1]	IN		V16		14	LVCMOS33*	3.300				NONE	NONE	
bin_in[0]	IN		V17		14	LVCMOS33*	3.300				NONE	NONE	
cat (8)	OUT				34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
cat[7]	OUT		W7		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
cat[6]	OUT		W6		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
cat[5]	OUT		U8		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
cat[4]	OUT		V8		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
cat[3]	OUT		U5		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
cat[2]	OUT		V5		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
cat[1]	OUT		U7		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	
cat[0]	OUT		V7		34	LVCMOS33*	3.300		12		NONE	FP_VTT_50	

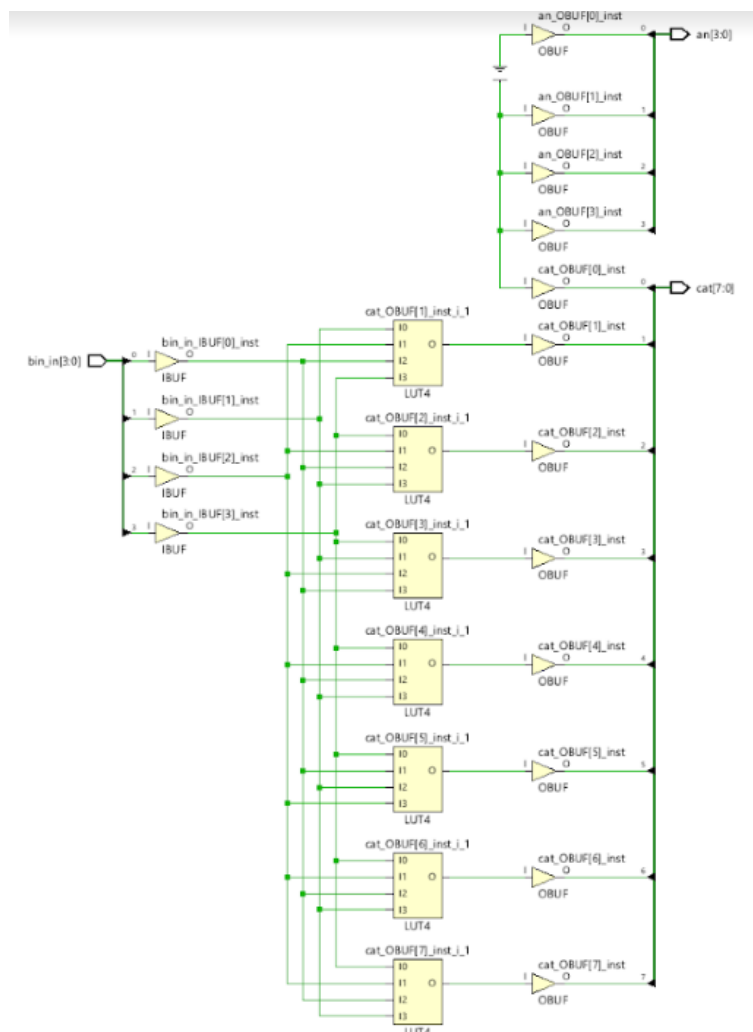
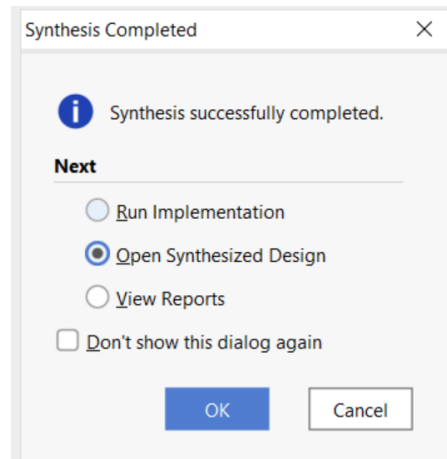
```

1  set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
2  set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
3  set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
5  set_property IOSTANDARD LVCMOS33 [get_ports {bin_in[3]}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {bin_in[2]}]
7  set_property IOSTANDARD LVCMOS33 [get_ports {bin_in[1]}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {bin_in[0]}]
9  set_property IOSTANDARD LVCMOS33 [get_ports {cat[7]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {cat[6]}]
11 set_property IOSTANDARD LVCMOS33 [get_ports {cat[5]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {cat[4]}]
13 set_property IOSTANDARD LVCMOS33 [get_ports {cat[3]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {cat[2]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {cat[1]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {cat[0]}]
17 set_property PACKAGE_PIN V17 [get_ports {bin_in[0]}]
18 set_property PACKAGE_PIN V16 [get_ports {bin_in[1]}]
19 set_property PACKAGE_PIN W16 [get_ports {bin_in[2]}]
20 set_property PACKAGE_PIN W17 [get_ports {bin_in[3]}]
21 set_property PACKAGE_PIN U2 [get_ports {an[0]}]
22 set_property PACKAGE_PIN U4 [get_ports {an[1]}]
23 set_property PACKAGE_PIN V4 [get_ports {an[2]}]
24 set_property PACKAGE_PIN W4 [get_ports {an[3]}]
25 set_property PACKAGE_PIN V7 [get_ports {cat[0]}]
26 set_property PACKAGE_PIN U7 [get_ports {cat[1]}]
27 set_property PACKAGE_PIN V5 [get_ports {cat[2]}]
28 set_property PACKAGE_PIN U5 [get_ports {cat[3]}]
29 set_property PACKAGE_PIN V8 [get_ports {cat[4]}]
30 set_property PACKAGE_PIN U8 [get_ports {cat[5]}]
31 set_property PACKAGE_PIN W6 [get_ports {cat[6]}]
32 set_property PACKAGE_PIN W7 [get_ports {cat[7]}]

```

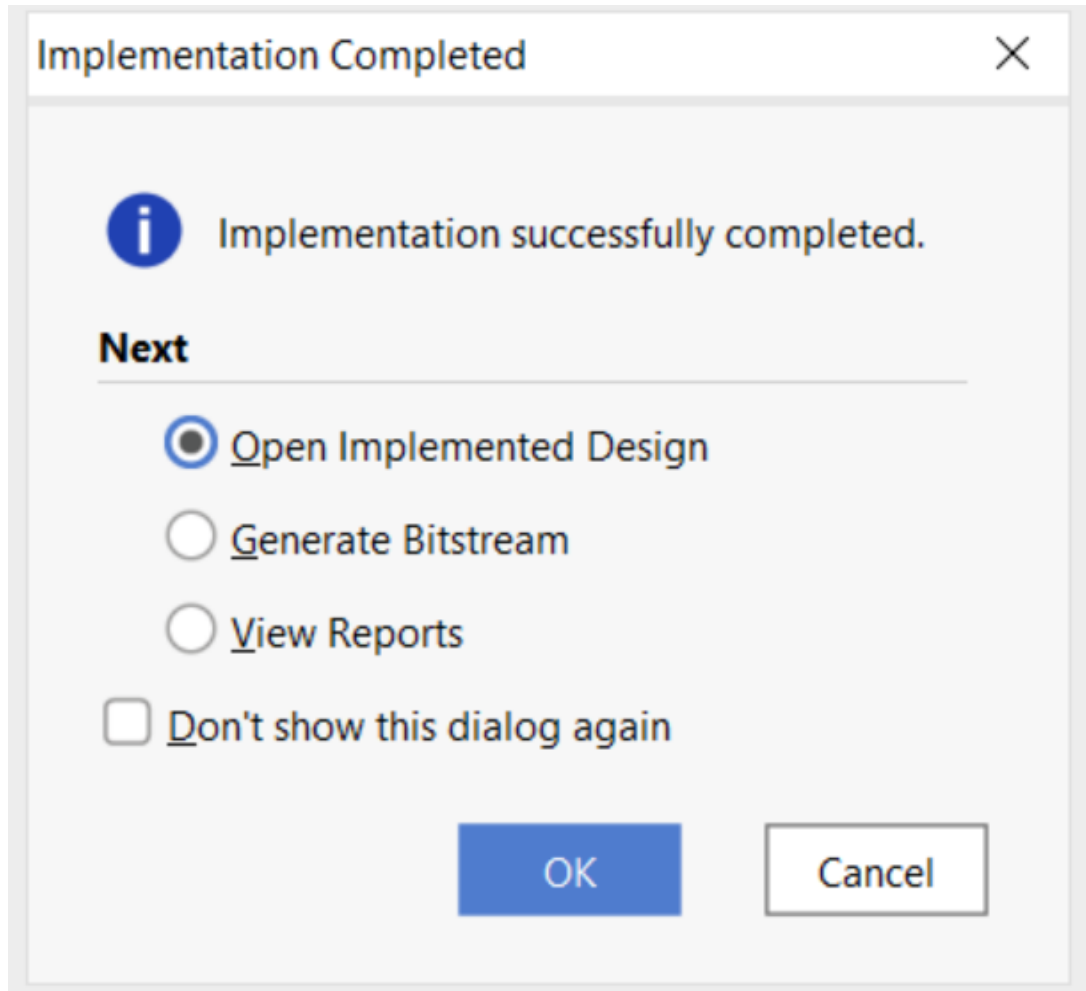
5. SÍNTESI

La síntesi consisteix en transformar el nostre disseny RTL a una representació a nivell de portes lògiques. Sintetitza el nostre disseny funcional per tal que es comporti com desitgem.



6. IMPLEMENTACIÓ

Aquest procés consisteix en implementar el netlist del disseny ja sintetitzat dins la nostra FPGA.



7. PROGRAM AND DEBUG

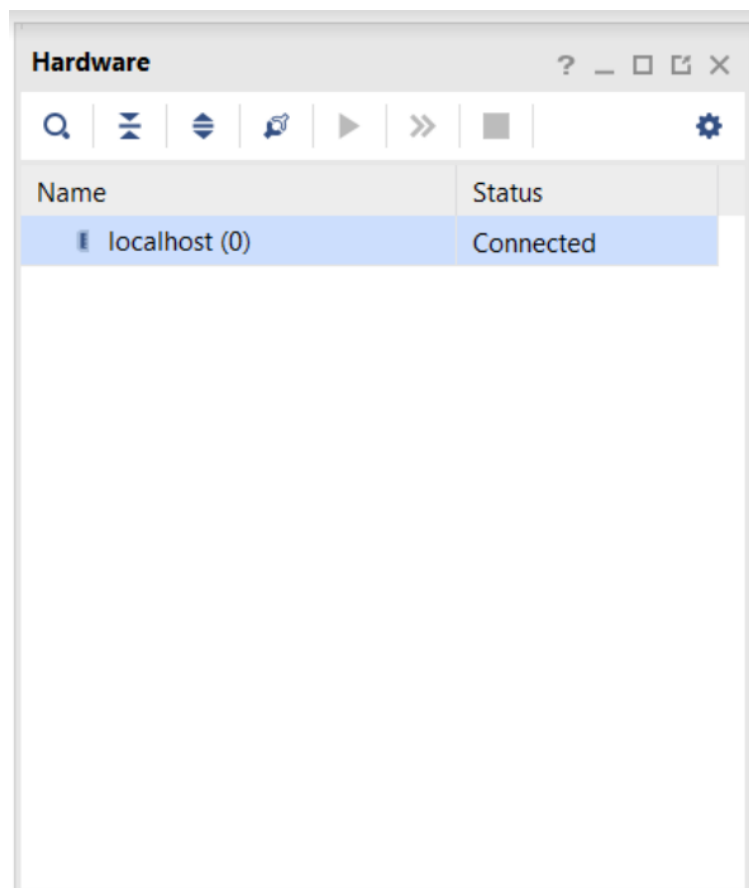
Aquest és l'últim pas i consisteix en connectar-nos via USB a la nostra placa per configurar la FPGA. Per fer-ho tenim dos opcions:

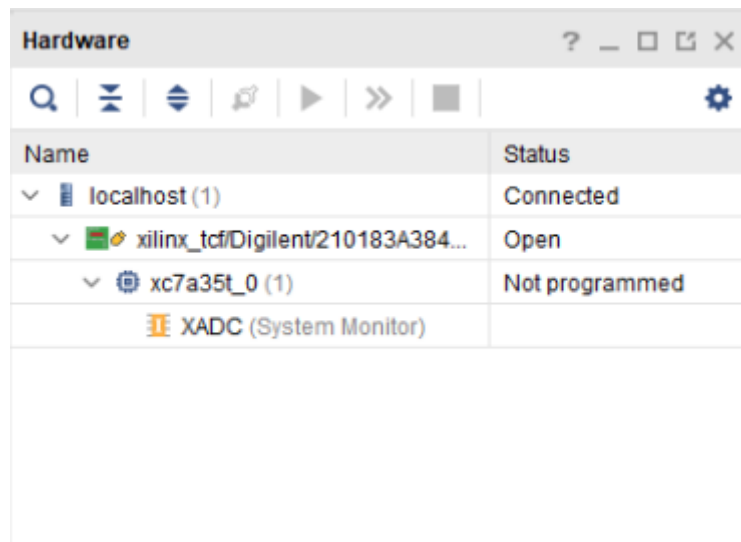
Configurant la FPGA directament des del JTAG amb un arxiu .bit. En aquest cas quan desconnectem l'alimentació de la placa la FPGA quedarà desconfigurada.

O gravant un arxiu .bin a la serial prom de la Basys 3. Així, cada vegada que alimentem la tarja, la FPGA es configurarà automàticament llegint aquesta memòria.

7.1 Configuració amb arxiu.bit

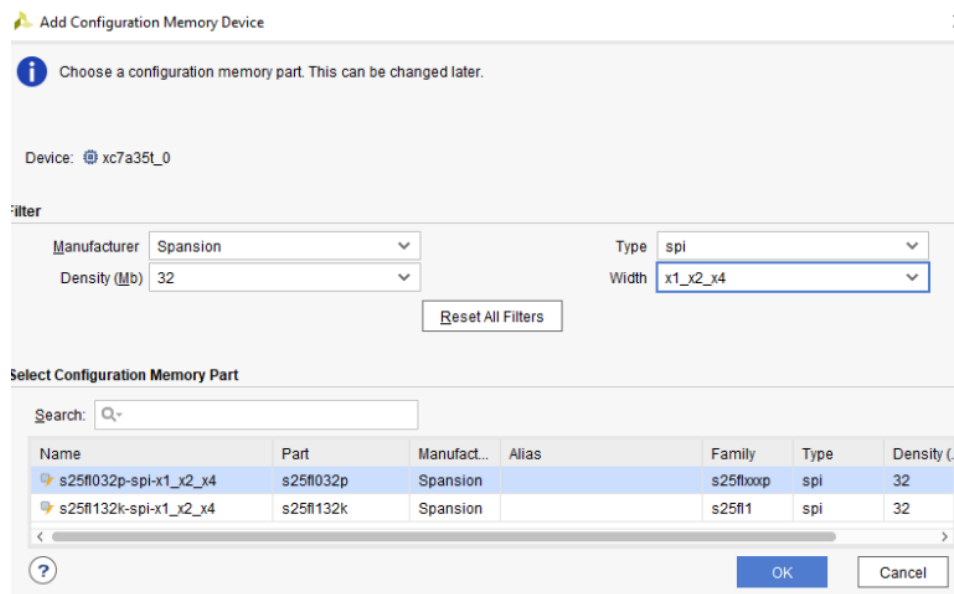
En el mateix apartat de “Program and Debug”, executarem l'acció “Generate Bitstream”. Quan acabi el procés s'obrirà la finestra per triar la següent acció a realitzar, triarem “Open Hardware Manager”.





7.2 Configuració amb arxiu .bin (gravació de la EEPROM sèrie)

Fem el mateix pas que en l'apartat anterior però clicem amb el botó dret sobre el nom del dispositiu i triem “Add Configuration Memory Device”.



 Program Configuration Memory Device ✕

Select a configuration file and set programming options. 

Memory Device:  s25fl032p-spi-x1_x2_x4 ...

Configuration file: ar_7seg/char_7seg.runs/impl_1/char_7seg.bin ✕ ...

PRM file: ...

State of non-config mem I/O pins: Pull-none ▾

Program Operations

Address Range: Configuration File Only ▾

☒ Erase

☐ Blank Check

☒ Program

☒ Verify

☐ Verify Checksum

SVF Options

☐ Create SVF Only (no program operations)

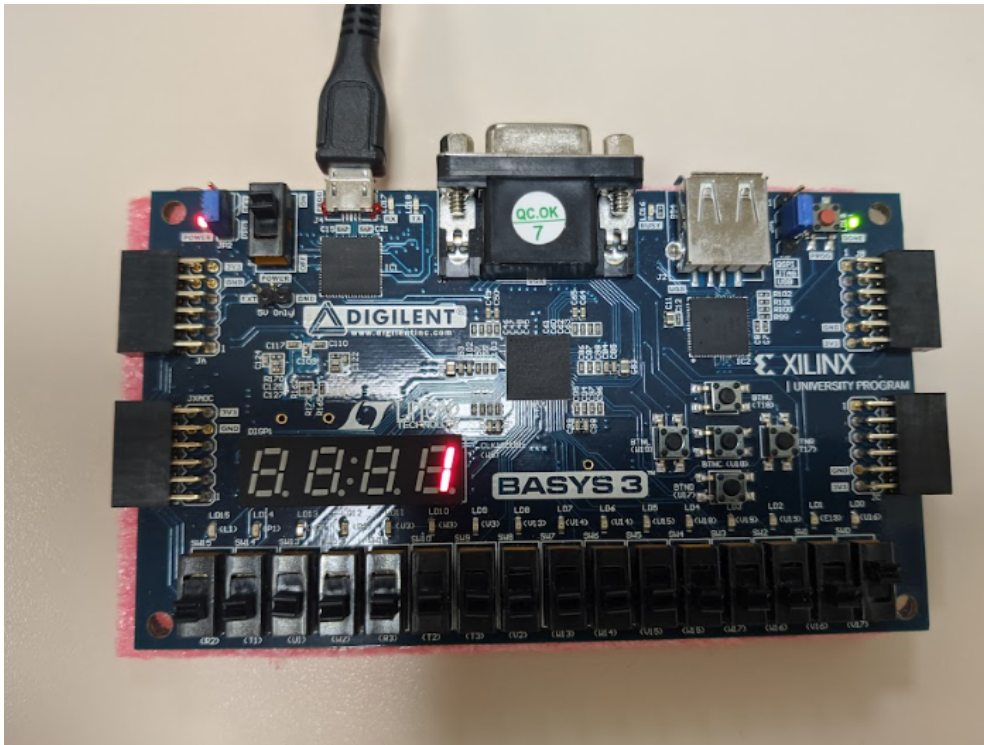
SVF File: ...

 OK Cancel Apply

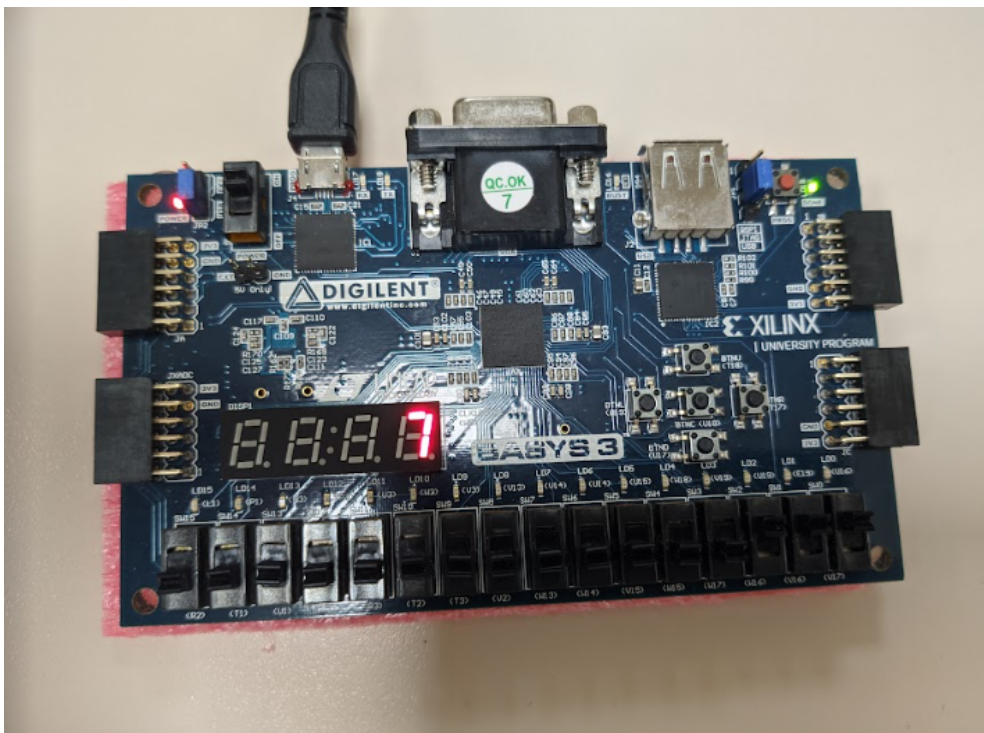
8. RESULTATS

Finalment podem veure el programa que hem creat reflexat a la nostra placa. El display s'il·lumina i mostra el número en base 16 (hexa) corresponent a l'entrada que li donem pels switch's.

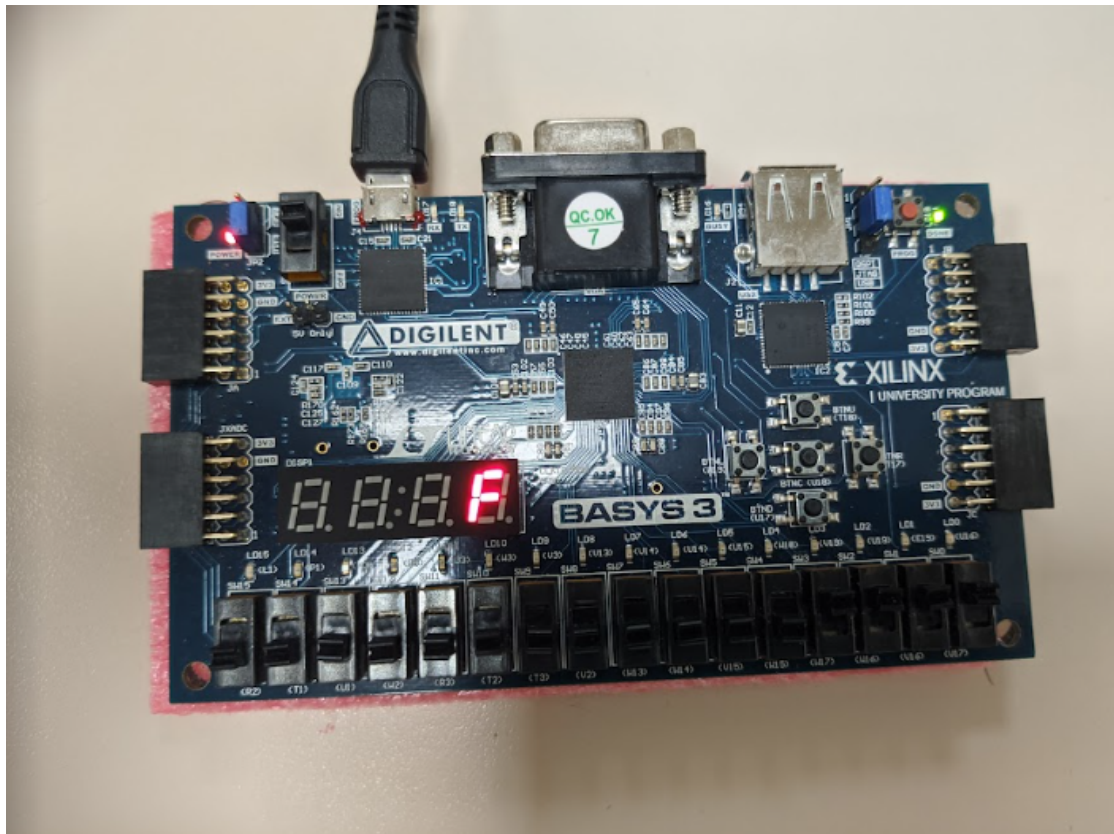
Switch en 0001



Switch en 0111



Switch en 1111



CONCLUSIONS

En l'execució d'aquesta pràctica hem après el funcionament bàsic de l'entorn de desenvolupament Vivado, creant un programa amb VHDL, sintetitzar-lo, implementar-lo i generant el corresponent binari. També hem après a simular el funcionament del nostre codi per simulació directa i mitjançant un *test bench* comprovant d'aquesta manera que el comportament del programa coincideix amb l'esperat. Finalment hem transferit el programa a la FPGA de la placa BASYS 3 Artix-7 i l'hem guardat a la memòria per tal que el programa romanguí a la placa al reconnectar l'alimentació.